

CUDA Graph 下确定性 Router GEMM 的多后端自动选核

脱敏分享版 | 本文讨论 vLLM、Triton、DeepGEMM 与 CUDA Graph 场景中的通用选核方法。所有模型、环境和服务信息均已匿名化。

结论先行

动态选核在 CUDA Graph 中不能理解为“replay 时试一下哪个 kernel 更快”。安全做法是在 eager warmup/正式 capture 前由主机侧 selector 完成 signature 构造、依赖检查、JIT、数值验证和独立 Graph preflight，缓存并冻结后端决定后再录制正式图。普通 replay 直接执行图中已固定的 custom op/backend，不再进入 Python selector，也不再做 signature lookup。

针对 decode 小 M 的 batch-invariant MoE Router GEMM，最终建立：

```
BF16: DeepGEMM -> Triton Full-K -> Persistent
FP16: Triton Full-K -> Persistent
FP32: Per-row Full-K -> Persistent
```

在一套 H200 kernel/模型 profile 中，代表性小 M microkernel 相对旧 persistent 路径提升约 2.95 倍；多层模型的 Router GEMM 聚合耗时下降 73.39%，step 内 device-kernel duration 之和的中位降幅为 6.16%，kernel 135/135、模型场景 20/20 逐位一致。另一套四卡 worker-engine 环境中，matched A/B 的 prefill/decode/generation throughput 提升约 3%–5%。两套环境和指标不混算；Auto 仍保持显式 opt-in，没有直接升级默认。

1. Router GEMM 为什么需要确定性

MoE Router 通常先计算：

```
logits = hidden @ gate_weight.T
experts = topk(logits)
```

Router GEMM 的极小浮点差异可能改变 Top-K 边界上的 expert 选择，进而影响 dispatch、expert 输出和后续 token。batch-invariant 模式追求相同请求不因 batch 组成而改变结果，因此需要避免：

- split-K 的跨 CTA 非固定归约；
- atomic accumulation；
- replay 间变化的后端；
- 未验证的 dtype promotion 或 fast-math；
- batch shape 变化导致的动态路径漂移。

确定性要求不是“kernel 名字相同”，而是输出所有权、K 维归约顺序和选择状态都稳定。

2. 旧 persistent 路径的问题

固定大 tile persistent kernel 能覆盖广泛 shape，也容易保持单一归约路径。但 decode 的 M 往往很小，大量线程和 tile 对应无效工作：



3.1 DeepGEMM 主路径

3.2 Triton Full-K fallback

3.3 Persistent 全覆盖 fallback

4. Dtype 需要不同的正确性合同

FP32 路径不应强求与旧 persistent 逐位相同，因为合法的固定归约顺序不同会改变末位舍入。更合理的合同是：对 FP64 reference 满足严格容差，并且相同 batch 与 Graph replay 自身稳定。

5. Capture-safe Selector 状态机

5.1 Warmup/正式 Capture 前可以做什么

这些步骤可能包含同步、编译或异常捕获，所以必须发生在正式 capture 前。其中 Graph preflight 是一次独立的预捕获/回放 smoke，不是在生产图 replay 期间选核。

5.2 Replay 中只能做什么

```
captured graph replay
-> fixed custom-op/backend node
-> fixed workspace/address contract
-> kernel launch
```

普通 replay 不会回到 Python selector，也不再查询 signature cache。replay 期间禁止：

- JIT；
- GPU→CPU 数值读取；
- 捕获异常后动态切后端；
- 根据当前输出重新 benchmark；
- 修改 workspace 地址或生命周期。

如果 warmup 或正式 capture 时 signature 未命中、缓存状态异常，应在图投入 replay 前 fail-fast，让上层重新走 preflight/capture，而不是在 replay 中静默改变图。replay 期间的 kernel 错误也应原样传播，不得被捕获后动态 fallback。

6. Signature 不能只包含 M

一个可靠 cache key 至少考虑：

```
(device capability,
dtype,
M/N/K,
strides/layout,
batch-invariant flag,
quantization mode,
graph mode,
kernel/version identity)
```

只按 **M** 缓存可能把不同 dtype、stride 或权重布局误认为同一 kernel 合同。依赖版本和 kernel identity 也应进入失效策略，避免升级后复用旧决定。

7. 四层验证

7.1 Kernel

- 多个小 M 与边界 shape；
- eager 与 Graph replay；
- batch 拆分/合并；
- 输出、Top-K 和重复运行确定性；
- 预检失败时的 fallback。

目标矩阵为 135/135 逐位通过。

7.2 多层模型

20 个受测场景均与基线逐位一致。Router GEMM 聚合中位耗时由 789.23 μ s/step 降至 210.01 μ s/step，下降 73.39%；trace 中每 step 所有 device-kernel duration 之和下降 2.77%–7.84%，中位为 6.16%。该指标是 kernel 时长求和，不是同步 wall time，也不是 GPU utilization。

7.3 Tensor Parallel

需要同时比较各 rank 的 Router 输出、expert 路由和最终 token。单 rank 正确不能证明 collective 或 batch 切分后仍一致。

7.4 独立四卡 worker-engine A/B

在另一套四卡 worker-engine 环境中，匹配输入、输出上限、缓存状态与采样配置后，代码生成型负载的 prefill/decode throughput 提升约 3.2%–3.4%，长 decode 的四个 matched 配置中 generation throughput 提升 4.66%–5.24%。只有采用确定性 Attention 的受测对照可声明 500/500 文本一致；含非确定性 stock Attention 的组别不能套用该结论。这组 worker-engine 数据与前述 H200 kernel/模型 profile 不是同一环境，不用它们做直接因果换算。

8. 为什么 local 2.95× 不能直接换算为 worker-engine 3%–5%

Router GEMM 只是整个生成路径的一小部分。根据 Amdahl 定律：

$$\text{overall_speedup} = 1 / ((1 - p) + p / \text{local_speedup})$$

即使 local kernel 接近 3×，Attention、expert GEMM、sampling、通信和框架开销仍保持不变。Amdahl 定律只说明局部优化会被未优化部分稀释；前述 local 与 worker-engine 数据来自不同环境，不能用后者反推前者的 Router 占比。真实归因必须依赖同环境的 matched profiler 与 wall-time 证据。

9. 为什么 Auto 仍不设默认

一个新后端成为默认，需要的不只是“比旧 persistent 快”：

- 相对已经优化过的 Full-K 是否仍有稳定、足够大的收益；
- 支持矩阵是否覆盖常见 dtype/shape；
- JIT、依赖和构建是否稳定；
- Graph capture、replay、恢复和 cache invalidation 是否闭环；
- 长跑与更多模型是否验证。

当前 Auto 使用显式 opt-in 灰度。即使 DeepGEMM 相对旧 persistent 很快，若相对成熟 Full-K 的增量未达到预设晋级门槛，也不应仓促替换默认。

10. 可复用检查表

- ☐ 后端是否避免 split-K、atomic 和非固定跨 CTA reduction？
- ☐ dtype 是否拥有各自明确的正确性合同？
- ☐ JIT、数值检查与 Graph preflight 是否全部在 capture 前？
- ☐ replay 中是否只执行已录制的固定 custom-op/backend 与 launch？
- ☐ cache key 是否覆盖 dtype、shape、stride、模式与版本？
- ☐ fallback 是否可观察、可测试、可回滚？
- ☐ 是否分开报告 kernel、模型、TP 和 worker-engine 结果？
- ☐ 是否避免把联合优化收益全部归因于 Router GEMM？

11. 总结

CUDA Graph 下的自动选核，本质上是一个状态机和正确性协议问题。高性能后端、低依赖 fallback 与全覆盖旧路径各司其职；所有可能同步或失败的工作前移到 capture 前，并把决定绑定到完整 signature，replay 才能保持固定且确定。H200 kernel/模型 profile 与独立四卡 worker-engine A/B 分别展示了局部和引擎层收益，但不应跨环境串成直接因果链；这正是分层验证和克制归因的重要性。